Project Handoff — SMA + Stigmergy Drone Swarm

June 25, 2026

What Was Fixed This Session

1. MAVSDK Cross-Read Bug (ROOT CAUSE FOUND & FIXED)

Drone 1 was reading Drone 0's telemetry because System() with no args shares a single mavsdk_server on default gRPC port 50051.

Fix: Pass unique gRPC port per drone:

```
# connection.py

async def connect_drone(port, grpc_port):

    drone = System(port=grpc_port)  # no mavsdk_server_address — lets it auto-launch

    await drone.connect(system_address=f'udp://:{port}')

# main.py

GRPC_PORTS = [50051, 50052]

drone = await connect_drone(DRONE_PORTS[i], GRPC_PORTS[i])
```

2. Two main.py bugs fixed

- Landing altitude was np.array([..., -3.0]) — should be 3.0 (positive-up system)
- safe_land_tasks.append(...) was outside the loop — only Drone 1 got a landing command

3. Sanity check threshold

Added post-takeoff position check, then removed it — both drones confirmed independent (NED-local origins differ by ~0.05m which is correct behavior, not a bug).

Best Run This Session (Run 6) ✅

Both drones: armed, took off, flew independently — FIRST TIME

Target: [40.0, 40.0, 15.0]

Final position: [39.42, 40.12, 11.90]

Final fitness: 3.161

Converged at: iteration 22 / 100

Altitude settled at 11.90 instead of 15.0 — local optimum, not a bug. Fixable by increasing Z_RANDOM from 0.15 to 0.25.

Sensor Stack Added (Not Yet Tested With Real Model)

Files created:

swarm_project/sensors/

├── __init__.py

├── lidar.py ← gz topic subprocess subscriber, min_range property

└── camera.py ← gz topic subprocess subscriber, frame counter + log

How it works:

- No gz Python bindings available — uses gz topic -e -t <topic> subprocess
- Each subscriber runs in a daemon thread, parses protobuf text output
- LidarSubscriber.min_range → float, thread-safe, readable from async code
- CameraSubscriber.frame_count → int, logs frame timestamps to logs/frames_N/

Gazebo topic names (when x500_sensors model is used):

/world/windy/model/x500_sensors_0/link/lidar_sensor_link/sensor/lidar/scan

/world/windy/model/x500_sensors_0/link/camera_link/sensor/camera/image

/world/windy/model/x500_sensors_1/link/lidar_sensor_link/sensor/lidar/scan

/world/windy/model/x500_sensors_1/link/camera_link/sensor/camera/image

fitness.py updated:

```
def fitness(positions, lidar_subscribers=None):
   # If lidar_subscribers provided → uses real LiDAR min_range
   # If None → falls back to simulated OBSTACLE_CENTER geometry check
```

main.py sensor wiring (already in current main.py):

```
lidars = [LidarSubscriber(i) for i in range(N_AGENTS)]
cameras = [CameraSubscriber(i) for i in range(N_AGENTS)]
for s in lidars + cameras: s.start()
await asyncio.sleep(2)
# fitness_vals = fitness(positions, lidars)  ← real LiDAR path
# for l in lidars: l.reset_range()          ← called each iteration
# for s in lidars + cameras: s.stop()       ← called after landing
```

---

x500_sensors Custom Model (Created, Not Yet Launch-Tested)

~/robotics_sim/PX4-Autopilot/Tools/simulation/gz/models/x500_sensors/

├── model.config

└── model.sdf ← x500 base + LiDAR (10x5 samples, 50m range) + mono_cam (1280x960 @ 30fps)

LiDAR upgraded from original single-point (1x1) to **10x5 scan pattern** — actually useful for obstacle detection.

Launch command when ready to test sensors:

```
# Drone 0
PX4_GZ_MODEL=x500_sensors \
PX4_GZ_MODEL_POSE="0,0,0.1,0,0,0" \
PX4_SIM_HOST_ADDR=127.0.0.1 \
make px4_sitl gz_x500_sensors

# Drone 1
PX4_SYS_AUTOSTART=4001 \
PX4_GZ_MODEL=x500_sensors \
PX4_GZ_MODEL_POSE="10,0,0.1,0,0,0" \
PX4_SIM_HOST_ADDR=127.0.0.1 \
./build/px4_sitl_default/bin/px4 -i 1
```

Current State of All Files

File	Status
connection.py	✅ Fixed — unique gRPC ports
main.py	✅ Fixed — sensors wired in, landing fixed
algorithm/fitness.py	✅ Updated — real LiDAR + simulated fallback
sensors/lidar.py	✅ Created — subprocess subscriber
sensors/camera.py	✅ Created — subprocess subscriber
models/x500_sensors/	✅ Created — not launch-tested yet

Immediate Next Steps

1. **Full kill/restart** (Drone 1 is in stale state from last run)
2. **Test plain x500 run first** to confirm algorithm still converges cleanly with sensors wired in (they'll show inf/0 — that's fine, falls back to simulated)

3. **Then restart with x500_sensors model** to get real LiDAR + camera topics
4. **Verify gz topics appear:** gz topic -l | grep -E 'lidar|camera'
5. **Then run the 3-condition paper experiment:**

Condition	PHEROMONE_WEIGHT	Jamming	Expected
A — Baseline	0.2	OFF	Best convergence
B — Jamming no stigmergy	0.0	ON	Drones stall in jam zone
C — Jamming + stigmergy	0.2	ON	Graceful degradation

Standard Launch Procedure (Every Run)

Step 0 — Kill everything

sudo pkill -9 -f px4; sudo pkill -9 -f gz; sudo pkill -9 -f mavsdk_server

sleep 5

ps aux | grep -E 'px4|gz|mavsdk' | grep -v grep # must be empty

Step 1 — Terminal 1: Drone 0

cd ~/robotics_sim/PX4-Autopilot

PX4_GZ_MODEL_POSE="0,0,0.1,0,0,0" PX4_SIM_HOST_ADDR=127.0.0.1 make px4_sitl gz_x500_windy

Step 2 — Terminal 2: Drone 1 (after Drone 0 shows Ready for takeoff!)

PX4_SYS_AUTOSTART=4001 PX4_GZ_MODEL=x500 PX4_GZ_MODEL_POSE="10,0,0.1,0,0,0" \

PX4_SIM_HOST_ADDR=127.0.0.1 ./build/px4_sitl_default/bin/px4 -i 1

Step 3 — In Drone 1 pxh shell

param set EKF2_MAG_TYPE 0

Step 4 — Terminal 3

cd ~/robotics_sim/swarm_project && python3 main.py

Known Warnings (All Harmless)

- Connection using udp:// is deprecated — safe, just use udpin:// eventually

- Ignore command 512 from 255/190 — QGC probing, ignore
- Preflight Fail: no heading reference — clears on arm after EKF2_MAG_TYPE 0
- LiDAR min_range=inf — expected when using plain x500 model without sensors